2600
2600, école de cybersécurité

MODULE
Noyau Windows

TP 03

Callbacks défensifs

Développement de drivers défensifs avec kernel callbacks

AUTEUR
Lionel Auroux

DATE
2026

DURÉE
5h

Sommaire

Objectifs . . . 3
Prérequis . . . 3
Rendu attendu . . . 4
Exercice 1 : Driver Process Monitor (2h) . . . 4
  Contexte . . . 4
  Objectifs . . . 4
  Indications . . . 4
  Vérification . . . 5
Exercice 2 : Driver Process Protector (2h) . . . 5
  Contexte . . . 5
  Objectifs . . . 5
  Indications . . . 6
    Structure d'enregistrement . . . 6
    Pre-operation callback . . . 6
  Vérification . . . 6
Exercice 3 (bonus) : Driver unifié monitoring + protection (1h) . . . 7
Barème indicatif . . . 7
Troubleshooting . . . 7

Objectifs

L'objectif de cet atelier est d'implémenter des drivers défensifs utilisant les kernel callbacks Windows :

- Implémenter un driver de monitoring de processus
- Implémenter un driver de protection de processus
- Combiner monitoring et protection dans un driver unifié



Fig. 1. – width:400px

Prérequis

- TP 01 et TP 02 complétés
- VM Windows en mode testsigning/debug avec WinDBG connecté
- Maîtrise de la compilation et du chargement de drivers (sc create/start/stop/delete)
- Maîtrise de DeviceIoControl (TP 02)
- Offsets `_EPROCESS` notés : `Token`, `ActiveProcessLinks`, `ImageFileName`

Rendu attendu

À la fin de ce TP, vous devez rendre une archive `tp03_NOM_PRENOM.zip` contenant :

Fichier	Description
`rapport_tp03.md`	Réponses aux questions, captures d'écran
`procmon/procmon.c`	Driver de monitoring de processus
`protector/protector.c`	Driver de protection de processus
`unified/unified.c`	(bonus) Driver combinant monitoring + protection
`unified/unified_ctl.c`	(bonus) Programme userland de contrôle

Le rapport doit contenir les réponses à toutes les questions numérotées et des captures d'écran de fonctionnement.

Exercice 1 : Driver Process Monitor (2h)

Contexte

Les kernel callbacks de notification de processus sont la base de tout EDR Windows. Vous allez implémenter un driver qui surveille la création et la destruction de processus en temps réel.

Objectifs

Écrire un driver `procmon.sys` qui :

1. Enregistre une callback via `PsSetCreateProcessNotifyRoutineEx` dans `DriverEntry`
2. Pour chaque **création** de processus, affiche via `DbgPrint` :
   - Le PID du nouveau processus
   - Le PID du processus parent (`CreateInfo->ParentProcessId`)
   - Le chemin de l'image (`CreateInfo->ImageFileName`)
   - La ligne de commande (`CreateInfo->CommandLine`)
3. Pour chaque **destruction** de processus, affiche le PID
4. Désenregistre proprement la callback dans `DriverUnload` (même fonction avec le flag `TRUE`)

Indications

- La callback reçoit un `PPS_CREATE_NOTIFY_INFO CreateInfo` qui est `NULL` en cas de destruction (et non-NULL en cas de création)

- Les chaînes `ImageFileName` et `CommandLine` sont des `PUNICODE_STRING` — les afficher avec le format `%wZ`
- Le cast du `HANDLE ProcessId` en `ULONG` pour l'affichage nécessite un double cast : `(ULONG)(ULONG_PTR)ProcessId`
- **Attention** : ne pas appeler de fonctions bloquantes dans la callback (elle s'exécute dans le contexte du processus créateur)

Vérification

Charger le driver, puis ouvrir/fermer des applications. Observer les logs dans DbgView.

Vérifier dans WinDBG que la callback est enregistrée :

```
code

kd> !callback
```

Q1 : Chargez votre driver, puis lancez et fermez `notepad.exe`. Faites une capture d'écran de DbgView montrant les événements de création et de destruction. Quel est le PID du processus parent de Notepad ? À quoi correspond-il ?

Q2 : Lancez `cmd.exe`, puis depuis ce cmd lancez `whoami.exe`. Observez la chaîne de parenté (PPID). Dessinez l'arbre des processus observé dans votre rapport.

Q3 : Lancez votre fuzzer du TP 01 (`ioctl_fuzzer.exe`) depuis un cmd. Observez les événements dans DbgView. Votre driver de monitoring voit-il la création du fuzzer ? Quelles informations le monitoring capture-t-il (chemin, ligne de commande, PID parent) ?

Exercice 2 : Driver Process Protector (2h)

Contexte

`ObRegisterCallbacks` permet de filtrer les opérations sur les objets kernel (processus, threads). Un EDR l'utilise pour empêcher un attaquant de tuer ses processus. Vous allez implémenter cette protection.

Objectifs

Écrire un driver `protector.sys` qui :

1. Prend un PID à protéger (hardcodé dans un `#define` pour simplifier, ou reçu via DeviceIoControl)
2. Enregistre un callback `ObRegisterCallbacks` sur les objets de type `*PsProcessType`

3. Pour chaque tentative d'ouverture de handle sur le processus protégé :
   - Retire le droit `PROCESS_TERMINATE` du masque `DesiredAccess`
   - Logue la tentative via `DbgPrint` (PID de l'appelant, droits demandés)
4. Le processus protégé ne peut plus être tué par `taskkill` ou le Task Manager

Indications

Structure d'enregistrement

`ObRegisterCallbacks` nécessite :

- Une structure `OB_OPERATION_REGISTRATION` spécifiant le type d'objet (`PsProcessType`), les opérations surveillées (`OB_OPERATION_HANDLE_CREATE | OB_OPERATION_HANDLE_DUPLICATE`) et la callback
- Une structure `OB_CALLBACK_REGISTRATION` contenant la version, le nombre d'opérations, une altitude (chaîne Unicode, ex: `"370000"`), et un pointeur vers l'`OB_OPERATION_REGISTRATION`
- Un handle retourné par `ObRegisterCallbacks` à conserver pour le désenregistrement via `ObUnRegisterCallbacks`

Pre-operation callback

Votre callback reçoit un `POB_PRE_OPERATION_INFORMATION` contenant :

- `OpInfo->Object` : le processus cible (castable en `PEPROCESS`)
- `OpInfo->Operation` : `OB_OPERATION_HANDLE_CREATE` ou `OB_OPERATION_HANDLE_DUPLICATE`
- `OpInfo->Parameters->CreateHandleInformation.DesiredAccess` : le masque de droits demandés

Pour retirer un droit : `DesiredAccess &= ~PROCESS_TERMINATE;`

Utilisez `PsGetProcessId` pour obtenir le PID à partir du `PEPROCESS`.

Vérification

1. Lancer `notepad.exe` et noter son PID.
2. Charger le driver avec le PID de notepad.
3. Tenter : `taskkill /PID <pid> /F` → doit échouer.
4. Tenter de fermer notepad via le Task Manager → doit échouer.
5. Décharger le driver → notepad redevient terminable.

Q4 : Testez les 5 étapes de vérification ci-dessus. Faites des captures d'écran de chaque étape et reportez les résultats dans votre rapport.

Q5 : Un attaquant avec un accès admin peut-il contourner cette protection ? Si oui, comment ? (Hint : pensez au BYOLD étudié dans le TP 01)

Q6 : Chargez simultanément votre `procmon.sys` (exercice 1) et `protector.sys`. Tentez de tuer le processus protégé. Que voyez-vous dans les logs de procmon ? Le monitoring détecte-t-il la tentative de kill même si elle est bloquée par le protector ?

Exercice 3 (bonus) : Driver unifié monitoring + protection (1h)

Combinez les exercices 1 et 2 dans un **driver unique** `unified.sys` qui :

- Monitore la création de processus (comme procmon)
- Protège un processus spécifié (comme protector)
- Expose un `DeviceIoControl` avec deux commandes :
  - `IOCTL_SET_PROTECTED_PID` : définir le PID à protéger
  - `IOCTL_GET_EVENTS` : récupérer les derniers événements loggés

Écrire un programme `unified_ctl.exe` :

```
code


unified_ctl.exe protect <pid>
unified_ctl.exe events
```

Q7 (bonus) : Ce driver unifié est un embryon d'EDR. Quels mécanismes supplémentaires faudrait-il ajouter pour en faire un EDR minimal fonctionnel ? Listez au moins 3 fonctionnalités manquantes avec le callback ou l'API kernel associé.

Barème indicatif

Élément	Points
Exercice 1 — Process Monitor fonctionnel (Q1–Q3)	8
Exercice 2 — Process Protector fonctionnel (Q4–Q6)	9
Bonus — Driver unifié (Q7)	3
Total	**20**

Troubleshooting

Problème	Solution
`PsSetCreateProcessNotifyRoutineEx` retourne `STATUS_ACCESS_DENIED`	Le driver doit avoir le flag `IMAGE_DLLCHARACTERISTICS_FORCE_INTEGRITY` dans son PE — ajouter `/integritycheck` dans les options du linker

`ObRegisterCallbacks` retourne `STATUS_ACCESS_DENIED`	Le driver doit être signé ou en testsigning. Vérifier aussi que l'altitude est une chaîne Unicode valide
BSOD `CRITICAL_STRUCTURE_CORRUPTION`	Vous avez probablement modifié une structure protégée par PatchGuard — utilisez uniquement les API officielles
La callback n'est jamais appelée	Vérifier que l'enregistrement a retourné `STATUS_SUCCESS`. Ajouter un `DbgPrint` de confirmation dans `DriverEntry`
`taskkill` fonctionne malgré la protection	Vérifier que vous filtrez aussi `OB_OPERATION_HANDLE_DUPLICATE` et pas seulement `HANDLE_CREATE`